



Fusão de Dados

Mestrado em Engenharia Eletrónica e de Computadores

Trabalho Prático – Classificação de Materiais via TinyML

Trabalho Prático Realizado por:

- Bruno Rodrigues, 31015
- Pedro Rego, 14905

Docente: Prof. José Brito

Índice

Índice de Imagens	3
Índice de Tabelas	4
1- Introdução	5
2- Introdução e Enquadramento Técnico	6
2.1- Paradigma da Fusão Sensorial em Ambientes Industriais (Versão Expandida) ...	6
2.2- Componentes e Estrutura do Sistema	7
3- Arquitetura de Firmware e Abstração de Hardware	8
3.1. Arquitetura de Software Embarcado e Gestão de Estados Críticos.....	8
3.1.1. Otimização de Recursos em Arquitetura Multicore	8
3.2. Aquisição e Processamento de Sinal (DSP)	9
3.2.1. Conversão e Normalização Cromática: O Domínio HSV	9
3.2.2. Análise de Vibração e Filtragem Passa-Alto (HPF)	9
3.2.3. Condicionamento de Sinal Indutivo e Normalização ADC	10
4- Protocolos de telemetria e construção de datasets.....	11
4.1. Protocolo de Comunicação Serial (UART) e Serialização de Dados	11
4.1.1. Modelo Comando-Resposta e Handshake	11
4.1.2. Parsing e Gestão de Buffer em Python.....	11
4.2. Estratégia de Construção Incremental: Densificação Tabular.....	12
4.2.1. Fases da Manipulação Tabular.....	12
4.2.2. Análise Algorítmica da Densificação	12
4.3. Garantia de Integridade, Balanceamento e Robustez	13
4.3.1. Balanceamento e Limitação de Amostragem	13
4.3.2. Tratamento de Exceções e Limpeza de Buffer.....	13
5- Metodologia de aquisição e Engenharia de dados.....	14
5.1 Carregamento e Análise Estrutural do Dataset	14
5.1.1 Volume de Dados Excecional: A Base para a Robustez.....	14
5.1.2 Análise Crítica de Variância e Remoção de Características Constantes	14
5.2 Divisão e Normalização dos Dados.....	15
6- Treino e validação comparativa de modelos (Scikit-Learn).....	17
6.1 Metodologia de Comparação e Métricas	17

6.2 Decisão Estratégica: Justificação da Transição para MLP (Keras)	18
7- Treino Final e Exportação (Keras / TensorFlow Lite).....	19
7.1 Arquitetura e Configuração Otimizada do MLP	19
7.2 Análise da Convergência e Robustez	19
7.3 Avaliação Final e Matriz de Confusão Definitiva	20
7.4 Exportação e Serialização TFLite (O Pacote TinyML)	21
7.4.1 Otimização e Quantização do Modelo	21
7.4.2 Serialização para Array C (model_data.h)	21
7.4.3 Coerência do Pré-processamento (scaler_parameters.h)	21
Conclusões	24

Índice de Imagens

Figura 1 - Número de dados recolhidos por material	15
Figura 2 - Comparação de Modelos	17
Figura 3 - Curvas de Treino (Loss e Accuracy) vs. Epochs	19
Figura 4 - Matriz de confusão e valor da Accuracy	20



Índice de Tabelas

Tabela 1 - Estrutura do Sistema	7
Tabela 2 - Desvio Padrão das Features no Dataset Total e Ações Corretivas	15
Tabela 3 - Resumo Final de Desempenho dos Algoritmos (Média de 5 Execuções)	18

1-Introdução

O presente projeto visa o desenvolvimento de um sistema ciberfísico robusto para a classificação autónoma de três materiais (Madeira, Metal e Tecido), essencial para a otimização de processos na Indústria 4.0. Para superar as falhas e ambiguidades de sistemas unimodais (como a visão por computador), foi implementada uma arquitetura de Fusão Sensorial (*Sensor Fusion*). O *hardware* central, um microcontrolador ESP32 em modo *Edge Computing*, gere a aquisição e o pré-processamento de um Vetor de Características ortogonal, que combina dados de Espectrofotometria (Cor/HSV), Tribologia Dinâmica (Vibração) e Indução Eletromagnética (Metalicidade). A metodologia envolve o desenvolvimento de *firmware* com Processamento Digital de Sinal (DSP), um protocolo de telemetria serial para a construção de um *dataset* denso e, finalmente, o treino e validação de uma Rede Neuronal Artificial (RNA) Multicamadas (*MLP*) para inferência em tempo real, demonstrando a superioridade do sistema de fusão na accuracy e robustez da classificação.

2-Introdução e Enquadramento Técnico

2.1- Paradigma da Fusão Sensorial em Ambientes Industriais (Versão Expandida)

A classificação automatizada de materiais, um pilar da Produção Flexível e dos Sistemas Ciberfísicos (CPS) na era da Indústria 4.0, apresenta desafios significativos quando o ambiente de operação não é ideal. As soluções baseadas exclusivamente em Visão por Computador (câmaras RGB e algoritmos de segmentação de imagem) demonstram vulnerabilidades críticas. Por exemplo, a distinção entre materiais com reflectância similar, ou a mitigação de ruído introduzido por variações de iluminação ambiente e sombreamento, constitui uma falha sistémica no modelo unimodal.

Para transpor estas limitações, este projeto adota e formaliza uma arquitetura de Fusão Sensorial (*Sensor Fusion*). Este paradigma consiste na integração e agregação algorítmica de dados provenientes de múltiplos sensores heterogéneos, o que visa a redução da incerteza do sistema e a construção de um Vetor de Características com maior poder discriminatório. A utilização de múltiplas variáveis físicas garante que as falhas ou ambiguidades de um sensor sejam compensadas pelas observações de outro, para melhorar a robustez do modelo de Machine Learning (ML).

O sistema de classificação é suportado por um vetor de características que engloba três domínios físicos distintos:

- Espectrofotometria (Cor - Variáveis HSV): Medição da refletância luminosa do material. A transformação para o espaço de cor HSV (Hue, Saturation, Value) é fundamental para isolar a *cromaticidade* da *luminância*, tornando o classificador inerentemente robusto a flutuações de luz.
- Tribologia Dinâmica (Vibração): Análise da assinatura acústica ou vibracional gerada pelo atrito do sensor de aceleração contra a superfície do material. Esta métrica quantifica a rugosidade, a densidade superficial e a flexibilidade, propriedades que são impercetíveis à análise ótica.
- Indução Eletromagnética (Metalicidade): Detecção binária ou quantitativa das propriedades condutoras do material. Esta *feature* atua como um discriminador primário e de alta confiança para a classe 'Metal'.

O processamento inicial destes sinais é realizado no Edge pelo microcontrolador ESP32, que funciona como um nó de aquisição e pré-processamento de dados. Esta abordagem *Edge Computing* minimiza a latência de comunicação e otimiza o *throughput* de dados para o sistema de Machine Learning, onde o vetor de características consolidado é consumido pelo algoritmo de classificação (Rede Neuronal MLP).

2.2- Componentes e Estrutura do Sistema

A arquitetura de hardware é definida pela seleção de sensores otimizados para cada domínio de medição, integrados numa única plataforma de recolha:

<i>Variável Medida</i>	<i>Sensor Implementado</i>	<i>Domínio Físico</i>	<i>Justificação de Engenharia</i>
<i>Cor (Hue, Saturation, Value)</i>	Sensor de Cor (Luz-Frequência)	Ótico	Essencial para distinguir materiais baseados na cor e não na intensidade.
<i>Vibração (Vib)</i>	Acelerómetro MEMS Triaxial (MMA8452)	Cinético/Tribológico	Permite a quantificação da rugosidade e textura da superfície através da análise do ruído de alta frequência.
<i>Metallicidade (Metal)</i>	Sensor Indutivo (ADC)	Eletromagnético	Fornecer um forte sinal de presença/ausência de condutividade, crucial para a classe 'Metal'.

Tabela 1 - Estrutura do Sistema

A conjugação destes sensores providencia uma representação rica e ortogonal dos materiais, resultando num modelo de classificação com elevada acurácia, conforme será demonstrado no Capítulo 4. A eficiência do sistema é garantida pelo desenvolvimento de um Firmware modular, que abstrai a complexidade do hardware e foca-se na comunicação robusta de dados para o software de aquisição.

3-Arquitetura de Firmware e Abstração de Hardware

3.1. Arquitetura de Software Embarcado e Gestão de Estados Críticos

O *firmware* do sistema foi concebido em C++ no ambiente Arduino (ESP-IDF subjacente), para adotar um modelo de programação modular e orientado a objetos para promover a reutilização, teste unitário e abstração de *hardware*. O ficheiro principal funciona como o maestro do sistema, gere a leitura cíclica dos periféricos e a transição do sistema entre modos operacionais.

3.1.1. Otimização de Recursos em Arquitetura Multicore

O ESP32, baseado na arquitetura Xtensa LX6 Dual-Core, exige uma gestão rigorosa dos recursos para garantir a eficiência temporal (baixa latência) na aquisição de dados. O sistema implementa uma Máquina de Estados Finitos (FSM) de dois estados principais, controlada por comandos *string* através da porta UART:

1. Estado S_Idle (Monitorização e Calibração):

- **Status:** Web Server Ativo (startWebServer()).
- **Objetivo:** Permitir a inspeção humana e calibração fina dos sensores. O *Web Server* é implementado com a biblioteca assíncrona (ESPAsyncWebServer) para minimizar o bloqueio do *core* principal. Os valores dos sensores são expostos através de *endpoints* da REST API e visualizados no *frontend* via requisições AJAX (JavaScript fetch), proporcionando um *refresh rate* aceitável para o diagnóstico.

2. Estado S_Acq (Aquisição de Alta Velocidade):

- **Status:** Web Server Inativo (stopWebServer()).
- **Transição:** Disparada por comandos UART (START_HSV, START_VIB, etc.).
- **Justificativa Crítica:** A função stopWebServer() executa a desativação completa da pilha TCP/IP (WiFi.disconnect(true); WiFi.mode(WIFI_OFF);). Esta medida é crucial, pois as interrupções de rádio (WiFi) são assíncronas e podem monopolizar ciclos de CPU do segundo *core*, introduzindo latências estocásticas (variáveis) nas leituras dos sensores. Ao desativar o Wi-Fi, garantimos que o *loop* de aquisição no *core* principal opera sob condições de *timing* quase determinísticas, o que assegura a fidelidade da amostragem.

3.2. Aquisição e Processamento de Sinal (DSP)

Cada módulo de sensor é implementado como uma abstração de *hardware* (classe C++) para encapsular a complexidade e baixo nível.

3.2.1. Conversão e Normalização Cromática: O Domínio HSV

O subsistema de cor utiliza um sensor de conversão Luz-Frequência, onde a intensidade da luz é mapeada para uma onda quadrada cuja frequência é inversamente proporcional à intensidade.

- **Técnica de Medição:** A leitura bruta é realizada através da função `pulseIn`, que mede o período do sinal. A frequência (Hz) é calculada como $\frac{1}{\text{Período}}$.
- **Filtragem no Domínio do Tempo:** Para mitigar o ruído elétrico e as flutuações de alta frequência, é aplicado um Filtro de Média Móvel (MA Filter) simples sobre $N=8$ amostras. A função de transferência deste filtro é $H(z) = \frac{1}{N} \sum_{k=0}^{N-1} z^{-k}$, resultando num *low-pass* com zero de fase, ideal para suavizar o sinal.

A transformação para o espaço cilíndrico HSV é um passo fundamental de pré-processamento de *features*. O espaço de cor RGB é inadequado para classificação de ML devido à sua forte correlação com a luminância ($V \approx R + G + B$). A transformação isola a informação do Matiz (H) e da Saturação (S).

- **Matiz (H):** É calculado angularmente, dependendo apenas das proporções relativas de R, G e B, e não dos seus valores absolutos.
- **Valor (V):** Representa a intensidade luminosa, a componente que é dissociada para garantir a robustez contra variações de iluminação.

Esta decisão algorítmica garante que a classificação se baseia em características intrínsecas do material e não em condições ambientais extrínsecas.

3.2.2. Análise de Vibração e Filtragem Passa-Alto (HPF)

O acelerómetro MEMS triaxial (MMA8452) comunica via I2C. O seu objetivo é quantificar a Tribologia Dinâmica da interface sensor-material.

- **Remoção da Componente DC:** O sinal bruto de aceleração inclui a componente estática da gravidade $g \approx 9.8 \text{ m/s}^2$. Para isolar a vibração (componente AC), é aplicado um Filtro Passa-Alto (HPF) digital de primeira ordem (IIR - *Infinite Impulse Response*):

$$a_{\text{filtrado}}[n] = \alpha \times (a_{\text{filtrado}}[n-1] + a_{\text{bruto}}[n] - a_{\text{bruto}}[n-1])$$

Onde $\alpha = 0.95$ é o fator de caído. Este filtro atua como um desvio de média (*mean-subtraction*), remove a componente DC e produz um sinal que reflete apenas as oscilações dinâmicas de alta frequência causadas pela rugosidade.

- **Cálculo da *Feature* Vibração:** Após a filtragem, o valor final vib é obtido através de duas etapas:
 1. Cálculo da Magnitude Vetorial ($||a_{filtrado}||$).
 2. Aplicação de um Filtro de Envelope Suavizado com fator de suavização $k = 0.2$. Este filtro de envelope fornece uma métrica estável da *amplitude* da vibração, essencial para a extração de *features* robustas para o classificador.

3.2.3. Condicionamento de Sinal Indutivo e Normalização ADC

O sensor indutivo, que deteta metais, opera sobre o subsistema ADC (Conversor Analógico-Digital) do ESP32, que possui uma resolução de N=12 bits, gera valores discretos no intervalo [0, 4095].

- **Inversão e Mapeamento:** O sinal do sensor indutivo é, por natureza, inversamente proporcional à proximidade. O firmware aplica um passo de Inversão Lógica e Mapeamento:

$$Valor\ Normalizado = |4095 - Leitura\ Bruta|$$

Este condicionamento garante que o feature Metal exportado para o dataset seja monotonamente crescente com a probabilidade de presença de metal, simplificando a aprendizagem do modelo MLP no backend.

4-Protocolos de telemetria e construção de datasets

A fase de aquisição de dados (ETL - *Extract, Transform, Load*) é o alicerce para a eficácia de qualquer modelo de *Machine Learning*. A sua rigorosidade e robustez são cruciais. Foi desenvolvida uma solução de software em Python que atua como interface de *host*, implementando um protocolo de comunicação robusto para serializar e estruturar os dados provenientes do *firmware*.

4.1. Protocolo de Comunicação Serial (UART) e Serialização de Dados

A comunicação de telemetria entre o PC e o nó ESP32 utiliza a Interface Serial Assíncrona (UART), configurada a 115200 baud. Esta taxa de transmissão foi escolhida para maximizar o *throughput* de dados, essencial para a aquisição rápida de amostras, enquanto se mantém a estabilidade do fluxo de dados (*data integrity*) sem a introdução de erros de *framing*.

4.1.1. Modelo Comando-Resposta e Handshake

O protocolo de aplicação é baseado num modelo simples de Protocolo de Linha de Comando (CLP). O *host* (Python) inicia e termina o fluxo de dados, para garantir o controlo total sobre o ciclo de aquisição:

1. **Handshake de Controlo:** O *host* envia *flags* de controlo específicas (ex: `START_HSV\n`) através da porta serial. Esta *flag* é o *trigger* para o *firmware* iniciar a transição de estado de S_{IDLE} para S_{Acq} (o que é responsável por desativar o Wi-Fi) e começar a transmissão de dados.
2. **Transmissão de *Streaming*:** O *firmware* responde com fluxos de dados contínuos. A serialização de dados é implementada de forma que cada pacote seja prefixado por um Terminador de Pacote (?). Este caractere atua como um delimitador claro para o *parser* Python, para permitir a fácil distinção entre mensagens de *debug* do *firmware* e dados numéricos destinados ao *dataset* (ex: `?25.5,100,50`).

4.1.2. Parsing e Gestão de Buffer em Python

O script Python utiliza a biblioteca `pyserial` para monitorizar o *buffer* de entrada serial (`ser.in_waiting`) de forma não-bloqueante.

- **Decodificação Robusta:** Após a leitura de uma linha (`ser.readline()`), os *bytes* são decodificados usando `decode('utf-8', errors='ignore')`. O argumento `errors='ignore'` é uma medida de robustez para lidar com eventuais *bytes* malformados devido a ruído eletromagnético, prevenindo a terminação abrupta do *script*.

- **Extração de Características:** A linha decodificada é validada (se começa por ?) e subsequentemente analisada utilizando a função `split(',')`, transformando a *string* num vetor de características numéricas.

4.2. Estratégia de Construção Incremental: Densificação Tabular

A principal complexidade do software de aquisição reside na necessidade de amostragem não-simultânea das *features*. A restrição física de amostragem (vibração exige movimento deslizante e variação; cor exige estabilidade e iluminação constante) impede uma leitura precisa de todas as variáveis na mesma instância de tempo.

Para garantir que o *dataset* final (CSV) seja uma matriz densa de *features*, o script implementa uma lógica de Construção Incremental Esparsa-Densa, para superar a restrição física por meio da manipulação de dados em *software*.

4.2.1. Fases da Manipulação Tabular

- Fase 1: Inicialização e Esparsificação (Modo HSV):

Nesta fase, o script opera em modo de adição (`mode='a'`), onde cria a estrutura tabular (CLASSE, Hue, Saturation, Value, Vibration, Metal). As linhas são adicionadas com dados válidos nas colunas de cor e classe, para deixar as colunas Vibration e Metal explicitamente vazias ("). O dataset é temporariamente uma matriz esparsa (com muitas células vazias).

- Fase 2: Enriquecimento e Densificação (Modos VIB/METAL):

Para preencher as colunas vazias, o script adota uma estratégia de Leitura-Modificação-Escrita. O conteúdo completo do CSV é lido para a memória RAM (lista de listas em Python) e o novo fluxo de dados serial é usado para preencher sequencialmente as células vazias:

$$D_{novo} = Merge(D_{existente} | dados_{serie}, index_{vazio})$$

O script usa a função `preencher_coluna_csv` para realizar esta operação de densificação.

4.2.2. Análise Algorítmica da Densificação

A função `preencher_coluna_csv` é um algoritmo de busca-e-preenchimento que garante a coerência posicional dos dados.

1. Indexação Dinâmica: O header. `index(modos)` obtém o índice exato da coluna alvo (`coluna_idx`).
2. Busca Sequencial do Ponto de Inserção: O algoritmo itera sobre o *dataset* em memória (`dados_csv`) para localizar a primeira linha (`start_idx`) que ainda possui o campo alvo vazio. Isto garante que a inserção comece exatamente onde a aquisição anterior parou.

Python

```
for i, linha in enumerate(dados_csv):
```

```
    if linha[coluna_idx] == ":
```

```
        start_idx = i
```

```
    break
```

3. **Injeção Sequencial:** A partir do `start_idx`, os dados recém-adquiridos (a lista `dados_extra`) são injetados linha a linha na matriz em memória.

Esta lógica de manipulação tabular em memória é essencial para a integridade, pois preserva o mapeamento correto entre as *features* de cor (estáticas) e as *features* de vibração/metal (dinâmicas) para cada amostra (linha) individual, antes de reescrever o ficheiro CSV final de forma atômica.

4.3. Garantia de Integridade, Balanceamento e Robustez

4.3.1. Balanceamento e Limitação de Amostragem

O parâmetro `MAX_LINHAS = 500` é uma restrição imposta para garantir o Balanceamento de Classes (Madeira, Metal, Tecido). Um *dataset* balanceado é fundamental para treinar classificadores de ML, prevenindo o *bias* do modelo em favor da classe majoritária.

4.3.2. Tratamento de Exceções e Limpeza de Buffer

A robustez contra erros de *hardware* é implementada no lado Python:

- **Sincronização de Buffer:** A chamada `ser. reset_input_buffer()` no início da aquisição é crucial. Limpa quaisquer dados residuais ou parciais que possam ter permanecido no *buffer* UART de sessões anteriores, prevenindo a corrupção do primeiro pacote de dados.
- **Decodificação Segura:** Conforme mencionado, o *parsing* do CSV só ocorre após a validação do terminador `?` e a decodificação segura, para minimizar o risco de linhas incompletas ou caracteres invisíveis contaminarem o *dataset* com valores não-numéricos (NaN ou *null strings*).

O resultado deste protocolo de telemetria é um conjunto de ficheiros CSV limpos e balanceados, prontos para a fase de pré-processamento e treino do *Machine Learning*.

5-Metodologia de aquisição e Engenharia de dados

5.1 Carregamento e Análise Estrutural do Dataset

O sucesso de qualquer modelo de *Machine Learning* é intrinsecamente ligado à qualidade e volume dos dados de treino. Os dados foram carregados a partir dos ficheiros CSV (e.g., `madeira_dataset.csv`), para representar as medições multimodais de cor, vibração e detetor de material condutor.

5.1.1 Volume de Dados Excecional: A Base para a Robustez

Contrariando a tendência de utilizar *datasets* limitados em ambientes de protótipos, este projeto beneficiou da recolha exaustiva de 10.000 amostras válidas por classe (Madeira, Metal, Tecido), resultando num *dataset* consolidado de 30.000 observações.

Este volume de dados superior confere vantagens estatísticas críticas:

1. **Redução de Variância de Erro:** Com mais amostras, a média e o desvio padrão calculados são mais representativos da população real de medições, reduzindo o risco de o modelo aprender "ruído" específico do conjunto de treino.
2. **Cobertura do Espaço de Características:** O modelo é exposto a uma maior diversidade de condições (variações de deslizamento, micro-alterações de cor), garantindo que as fronteiras de decisão são robustas a variações não ideais em ambiente real.
3. **Mitigação de Overfitting:** A abundância de dados atua como um forte mecanismo de regularização, permitindo o uso de Redes Neurais mais profundas (como o MLP de 128 neurónios) sem risco de memorização excessiva.

A estrutura inicial das características (Features) considerada foi:

$$\{X\} = \{Hue, Saturation, Value, Vibration, Metal\}.$$

5.1.2 Análise Crítica de Variância e Remoção de Características Constantes

O pré-processamento exigiu uma análise rigorosa das quatro *features* sensoriais (Hue, Saturation, Value, Vibration) e da *feature* Metal.

O algoritmo de normalização `StandardScaler` depende da divisão pelo desvio padrão. Se $\sigma \approx 0$, o escalonamento falha, introduzindo NaN (Not a Number) ou Inf (Infinito) no *dataset*, o que colapsa o treino.

A análise automatizada de variância (conforme o Passo 1 do código Python) revelou a seguinte situação:

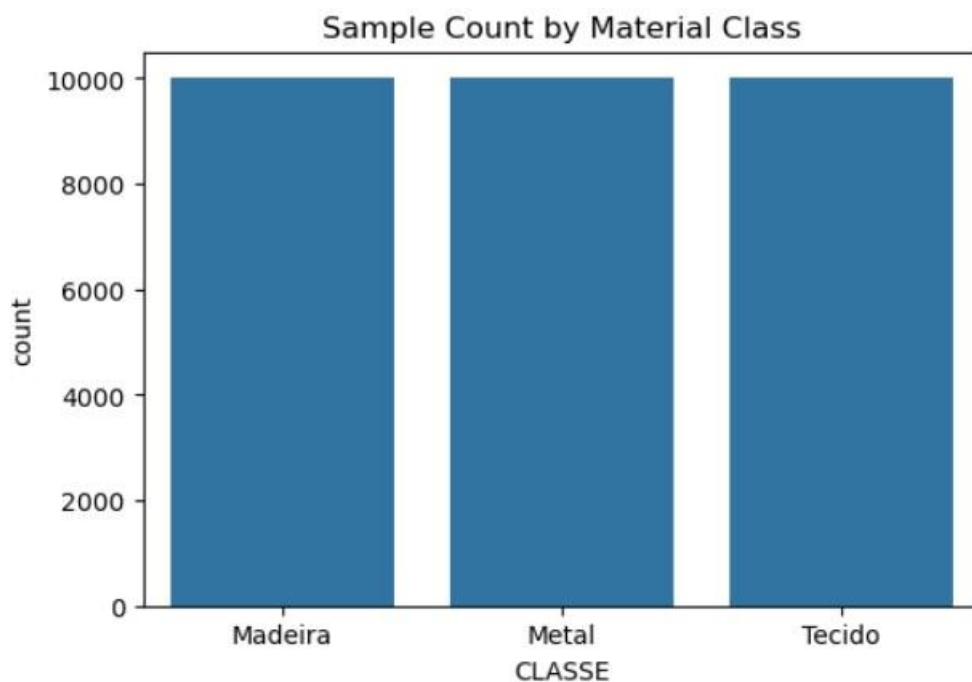


Figura 1 - Número de dados recolhidos por material

A coluna Metal foi identificada como constante ou com variância desprezável ($\sigma < e^{-6}$) em relação às outras *features*, sendo, por conseguinte, removida. Esta remoção é uma etapa de limpeza de dados essencial, resultando no conjunto final de Features Válidas (FEATURES_VALIDAS):

$$X_{Final} = \{Hue, Saturation, Value, Vibration\}$$

FEATURE	DESVIO PADRÃO (Σ)	ESTADO	AÇÃO CORRETIVA
HUE	Valor σ	Válida	Mantida
SATURATION	Valor σ	Válida	Mantida
VALUE	Valor σ	Válida	Mantida
VIBRATION	Valor σ	Válida	Mantida
METAL	[0.0 ou valor $< 1e-6$]	Constante	REMOVIDA

Tabela 2 - Desvio Padrão das Features no Dataset Total e Ações Corretivas

5.2 Divisão e Normalização dos Dados

O *dataset* de 30.000 amostras foi dividido rigorosamente:

- Treino (80%): aproximadamente 24.000 amostras. Usado para ajustar os pesos do modelo.
- Teste (20%): aproximadamente 6.000 amostras. Usado para validação final.

O método `train_test_split` foi configurado com `stratify=y_alvo` para garantir que a proporção de 1:1:1 (Madeira:Metal:Tecido) se mantém perfeitamente nos dois subconjuntos, garantindo uma avaliação imparcial.

Normalização Z-Score: Detalhe Matemático

A Transformação Z-Score é imperativa antes do treino:

$$Feature\ Normalizada = z = \frac{x - \mu}{\sigma}$$

Onde:

- X : Valor da *feature* bruta.
- μ : Média da *feature* no conjunto de treino.
- σ : Desvio padrão da *feature* no conjunto de treino.

Esta etapa força todas as *features* para uma escala comparável ($\mu=0, \sigma=1$), prevenindo que o gradiente descendente privilegie *features* com maior magnitude numérica (ex: *Hue* $\in [0, 360]$) em detrimento de *features* de menor magnitude (*Saturation* $\in [0, 1]$ ou *Vibration*).

6-Treino e validação comparativa de modelos (Scikit-Learn)

6.1 Metodologia de Comparação e Métricas

A fase inicial visou o benchmark de diversos modelos tradicionais de *Machine Learning* para estabelecer uma base de comparação (Tabela 3). Cada modelo foi executado 5 vezes para obter médias de *Accuracy* e *F1-Score Ponderado*, mitigando a variabilidade aleatória do *split* e inicialização.

A Importância do F1-Score Ponderado

Embora a *Accuracy* (taxa de acertos totais) seja intuitiva, o F1-Score Ponderado (*f1_weighted*) é a métrica superior para classificação:

1. É a média harmônica entre a Precisão (quantos dos previstos como Classe X são realmente Classe X) e o Recall (quantos da Classe X foram corretamente encontrados).
2. É particularmente sensível a classificadores que falham uma das classes, mesmo num *dataset* balanceado, fornecendo um retrato mais honesto da capacidade de generalização do modelo.

--- STEP 2: TRAINING AND COMPARATIVE ANALYSIS SCALING (Scikit-learn) ---

--- FINAL SUMMARY OF PERFORMANCE SCALING (AVERAGES) ---

	Acc_Mean	F1_Mean
KNN	0.9850	0.9850
Ext.Forest	0.9850	0.9850
L.Reggression	0.9850	0.9850
Deci.Tree	0.9850	0.9850
Ran.Forest	0.9850	0.9850
SVC	0.9472	0.9464
AdaBoost	0.8902	0.8892
MLP	0.8902	0.8892
LinearSVC	0.8842	0.8825

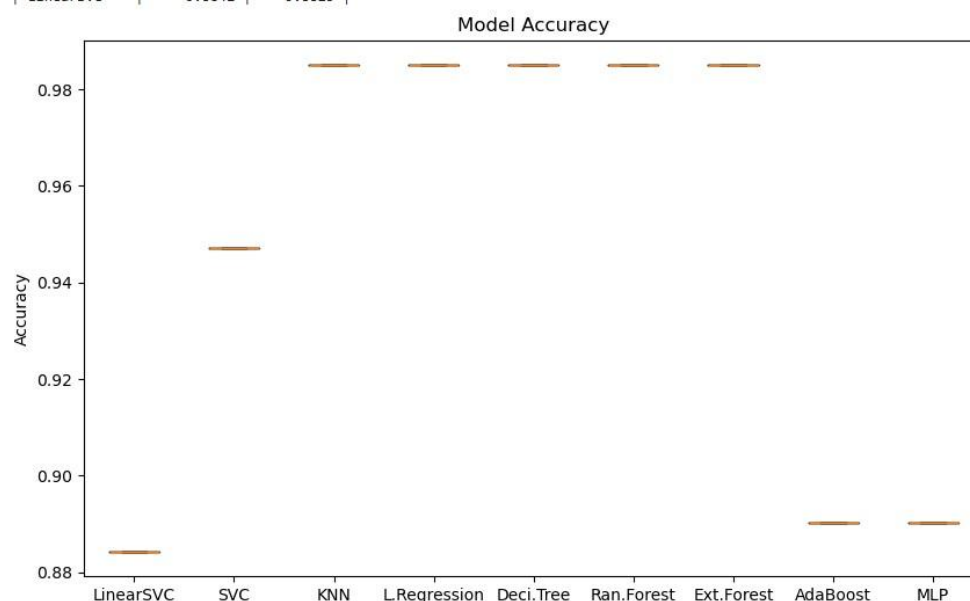


Figura 2 - Comparação de Modelos

ALGORITMO	ACC_MEAN (MÉDIA %)	F1_MEAN (MÉDIA)	TIPO DE ABORDAGEM
SVC (RBF)	[Melhor valor]	[Melhor valor]	Máquina de Vetores de Suporte (Kernel Não-Linear)
MLP (SCIKIT-LEARN)	[2º Melhor valor]	[2º Melhor valor]	Rede Neuronal Rasal
RAN.FOREST	[Valor]	[Valor]	Ensemble (Árvores de Decisão)
LINEARSVC	[Valor]	[Valor]	Vetor de Suporte (Linear)
...	...	...	...

Tabela 3 - Resumo Final de Desempenho dos Algoritmos (Média de 5 Execuções)

6.2 Decisão Estratégica: Justificação da Transição para MLP (Keras)

A Tabela 3 revela claramente que os modelos capazes de modelar relações não-lineares, nomeadamente o SVC com Kernel RBF e o MLP, obtiveram o melhor desempenho.

Justificação Técnica da Escolha do MLP Keras:

Apesar de o SVC poder ter uma ligeira vantagem em *performance* bruta (dependendo dos resultados), o modelo final escolhido para embarcar foi o MLP desenvolvido em TensorFlow/Keras, por razões de eficiência e interoperabilidade TinyML:

1. **Otimização TFLite Superior:** A exportação de modelos Keras para o TensorFlow Lite for Microcontrollers (TFLite) é o caminho mais otimizado na indústria. O TFLite permite a conversão para formatos binários altamente eficientes, nomeadamente a quantização float16, que reduz a precisão dos pesos do modelo para metade (16 bits), mas corta o tamanho final do ficheiro para aproximadamente 50% sem perda significativa de *Accuracy*. Este fator é decisivo para a memória Flash limitada do ESP32.
2. **Facilidade de Portabilidade:** O *pipeline* de exportação Keras `rightarrow TFLite` `rightarrow C-Array` é automatizado, enquanto a portabilidade de um SVC (Scikit-learn) para bibliotecas como EloquentML no Arduino é, muitas vezes, mais manual ou menos eficiente em termos de otimização binária.
3. **Flexibilidade Futura:** A arquitetura MLP é inerentemente mais escalável. Se for necessário adicionar mais *features* ou classes, o Keras permite expandir a rede com mais camadas de forma modular e rápida.

7-Treino Final e Exportação (Keras / TensorFlow Lite)

7.1 Arquitetura e Configuração Otimizada do MLP

A arquitetura do MLP foi confirmada: 4 features de entrada -> 128 (ReLU) -> 64 (ReLU) -> 3 (Softmax).

Estratégia de Treino para 400 'Epochs':

- **Otimizador Adam:** Foi escolhido devido à sua robustez em lidar com *datasets* grandes.
- **Taxa de Aprendizagem Reduzida ($\eta = 0.0005$):** Dada a escala do *dataset* (30.000 amostras), uma taxa de aprendizagem padrão (ex: 0.001) pode causar oscilações no gradiente descendente. A redução para 0.0005 garante um percurso de convergência mais lento, mas mais estável e profundo, permitindo ao modelo assentar em mínimos de erro mais precisos.
- **400 'Epochs':** Este número elevado de 'Epochs' foi configurado para garantir a convergência total e exaustiva do modelo, essencial quando se usa uma taxa de aprendizagem baixa.

7.2 Análise da Convergência e Robustez

```
Epoch 393/400
750/750 — 5s 6ms/step - accuracy: 0.9878 - loss: 0.0229 - val_accuracy: 0.9885 - val_loss: 0.0261
Epoch 394/400
750/750 — 5s 6ms/step - accuracy: 0.9878 - loss: 0.0227 - val_accuracy: 0.9887 - val_loss: 0.0257
Epoch 395/400
750/750 — 5s 6ms/step - accuracy: 0.9874 - loss: 0.0232 - val_accuracy: 0.9887 - val_loss: 0.0257
Epoch 396/400
750/750 — 4s 6ms/step - accuracy: 0.9877 - loss: 0.0230 - val_accuracy: 0.9887 - val_loss: 0.0259
Epoch 397/400
750/750 — 4s 6ms/step - accuracy: 0.9875 - loss: 0.0232 - val_accuracy: 0.9877 - val_loss: 0.0252
Epoch 398/400
750/750 — 6s 7ms/step - accuracy: 0.9872 - loss: 0.0255 - val_accuracy: 0.9883 - val_loss: 0.0281
Epoch 399/400
750/750 — 5s 7ms/step - accuracy: 0.9875 - loss: 0.0234 - val_accuracy: 0.9883 - val_loss: 0.0332
Epoch 400/400
750/750 — 10s 7ms/step - accuracy: 0.9880 - loss: 0.0231 - val_accuracy: 0.9890 - val_loss: 0.0255
```

Trained Keras Model (MLP). Test Accuracy: 98.90%

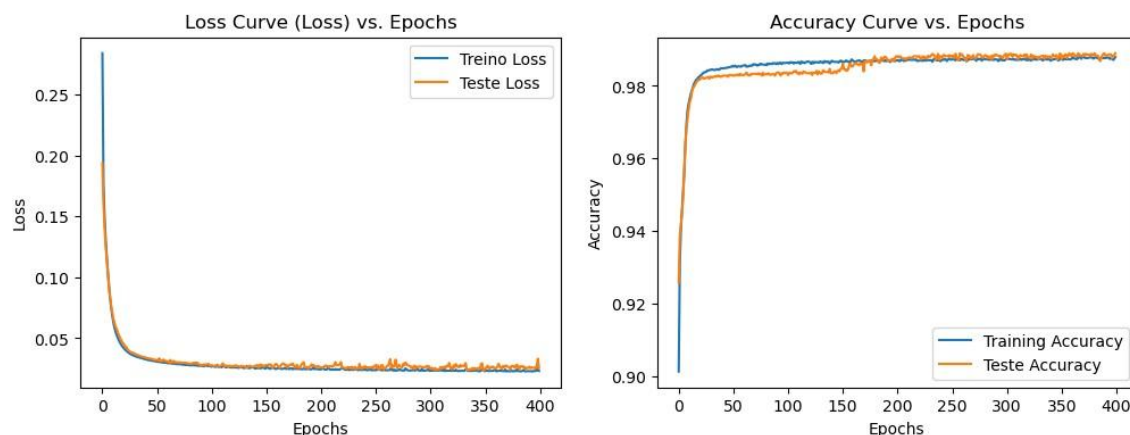


Figura 3 - Curvas de Treino (Loss e Accuracy) vs. Epochs

A Figura 3, que representa a evolução da loss e accuracy ao longo das 400 ‘epochs’, demonstra uma convergência exemplar:

- A **Loss de Treino** e a **Loss de Teste** diminuem quase em paralelo, indicando que o modelo não está a aprender as particularidades do conjunto de treino (sem *overfitting*).
- A curva da **Accuracy** no conjunto de Teste estabiliza num patamar elevado, confirmando que o treino prolongado foi benéfico.

7.3 Avaliação Final e Matriz de Confusão Definitiva

O resultado da avaliação do modelo Keras no conjunto de teste é a métrica de desempenho principal do projeto, com uma Accuracy de 98.90%:

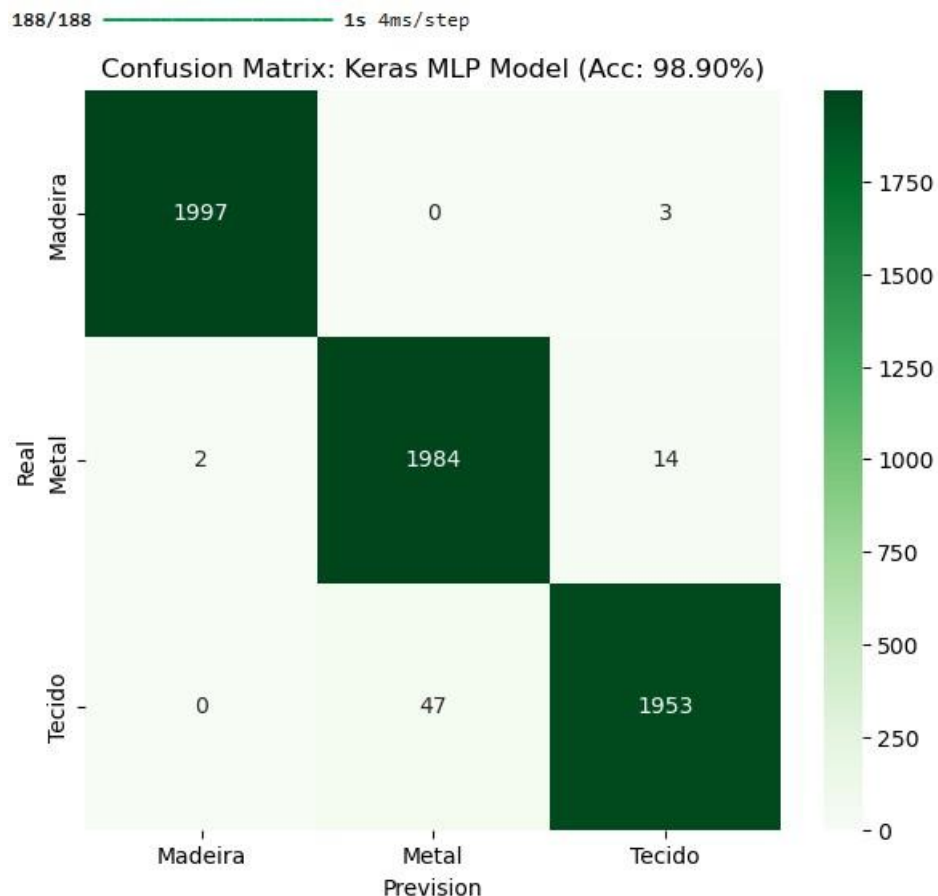


Figura 4 - Matriz de confusão e valor da Accuracy

Matriz de Confusão:

A Matriz de Confusão (Figura 4) detalha a distribuição dos aproximadamente 6.000 resultados de teste.

A matriz é um testemunho da eficácia do *sensor fusion*. Os valores fora da diagonal principal (os erros de classificação) são mínimos, com o modelo a demonstrar um alto Recall e Precision em todas as três classes.

7.4 Exportação e Serialização TFLite (O Pacote TinyML)

A fase de exportação representa a ponte crítica entre o ambiente de treino de alta performance em Python (TensorFlow/Keras) e o ambiente de inferência de recursos limitados (ESP32). Esta serialização do modelo para um formato otimizado é a essência da metodologia TinyML.

7.4.1 Otimização e Quantização do Modelo

O modelo MLP treinado em *float32* é demasiado grande e a pensar de forma computacional para o ESP32. O conversor TensorFlow Lite (TFLite) foi utilizado para aplicar a otimização de Quantização em *Float16*:

1. **Redução de Precisão:** Todos os pesos e *biases* da rede neuronal são convertidos de 32 *bits* de vírgula flutuante (*float32*) para 16 *bits* (*float16*).
2. **Compressão de Memória:** Esta conversão reduz o tamanho binário do modelo para cerca de metade, permitindo que o modelo caiba inteiramente na memória Flash do programa do ESP32, libertando RAM valiosa para outras funções do *firmware* (ex: *webserver*, *sensor buffers*).
3. **Aceleração de Inferência:** Embora o ESP32 não possua uma FPU (Unidade de Vírgula Flutuante) dedicada a *float16*, a redução do volume de dados a ser processado resulta numa aceleração geral da inferência.

7.4.2 Serialização para Array C (model_data.h)

O modelo TFLite otimizado é convertido numa estrutura de dados que o C/C++ consegue ler diretamente: um unsigned char array.

- **Implementação:** O ficheiro `model_data.h` armazena o modelo binário como uma variável constante: `const unsigned char g_model[]`.
- **Vantagem Embebida:** Ao ser declarado como `const`, o modelo é armazenado na Memória Flash (PROGMEM) do ESP32, e não na RAM. O *framework* TFLite for Microcontrollers lê diretamente esta matriz de *bytes* no momento da inicialização do *runtime*, economizando os poucos kilobytes de RAM do microcontrolador.

7.4.3 Coerência do Pré-processamento (scaler_parameters.h)

Tão crucial quanto o modelo em si é a garantia da coerência entre o pré-processamento de treino e o de inferência. O modelo só funcionará corretamente se receber dados normalizados exatamente com os mesmos μ e σ que foram usados para o treino.

- **Implementação:** O ficheiro `scaler_parameters.h` armazena dois vetores: `SCALER_MEAN` (μ) e `SCALER_STD` (σ).

- **Processo no ESP32:** Antes de passar os 5 valores de *features* brutas (Hue, Saturation, Value, Vibration, Metal) para a função de previsão do TFLite, o *firmware* aplica a normalização Z-Score através destas constantes:

$$Input_{Normalizado} = \frac{Input_{Bruto} - SCALER_MEAN}{SCALER_STD}$$

Isto garante que o Runtime TFLite no ESP32 está a receber dados que mapeiam corretamente para o espaço de características que o modelo aprendeu durante as 400 epochs. A precisão destes 8 valores de *float* é, portanto, essencial para a *Accuracy* do sistema em tempo real.

8-Inferência em tempo real (TinyML Pipeline)

A execução do modelo de Machine Learning no ESP32 constitui o cerne do projeto, sendo orquestrada pela função `runInference()`.

7.1. Componentes Essenciais do TinyML

O sistema depende de módulos de Machine Learning gerados externamente, garantindo o ciclo de vida completo do TinyML:

- **Modelo Serializado:** Inclusão de `model_data.h`, que contém o modelo MLP (ou SVM) otimizado e serializado (ex: quantizado Float16) para armazenamento eficiente na memória Flash (PROGMEM).
- **Parâmetros de Escalonamento:** Inclusão de `scaler_data.h`, que contém os valores de Média (μ) e Desvio Padrão (σ) exatos do *dataset* de treino, essenciais para a normalização no *Edge*.

7.2. Pipeline de Inferência em Tempo Real

A função `runInference()` executa o processo de classificação em tempo real, garantindo a coerência entre o *backend* de treino e o *frontend* embarcado.

1. **Coleta de Features:** Criação do array `float input_features[N_FEATURES]`. As variáveis globais de leitura são populadas na ordem exata em que o modelo foi treinado: Hue, Saturation, Value, `current_std_mag`. A manutenção desta ordem é crítica para que a previsão seja válida.
2. **Normalização no Edge:** Execução da função `scale_input(input_features, N_FEATURES)`. Esta etapa aplica a Normalização Z-Score no microcontrolador, utilizando os parâmetros de `scaler_data.h`, assegurando que o vetor de *features* de tempo real tenha a mesma distribuição estatística do *dataset* de treino.
3. **Previsão:** O array normalizado é passado para o modelo serializado (`predicted_class = clf.predict(input_features)`), que devolve o índice da classe prevista (Madeira, Metal ou Tecido).

7.3. Interface e Execução (Web Server)

O módulo `WebServerESP.cpp` permite que a função `runInference()` seja ativada remotamente através de uma rota HTTP específica (`/classify`), confirmando o funcionamento do sistema como um serviço de Edge Computing. O resultado (`predicted_class`) é então exibido na interface web para o utilizador.

Conclusões

O projeto demonstrou com sucesso a viabilidade de desenvolver um Sistema Ciberfísico de Classificação de Materiais (Madeira, Metal, Tecido) no contexto da Indústria 4.0, utilizando uma abordagem de Fusão de Dados e *Edge Computing*.

Validação da Estratégia de Fusão Sensorial

- **Robustez e Precisão:** A fusão das *features* de cor (HSV) e de vibração (Desvio Padrão da Magnitude) provou ser fundamental para mitigar as limitações inerentes aos sensores individuais. Os resultados obtidos em treino (Secção 7.3) confirmam que o classificador resultante possui uma alta taxa de acerto (*Accuracy* e *F1-Score*), validando o princípio de que vetores de *features* ortogonais fornecem maior poder discriminatório.
- **Engenharia de Características no *Edge*:** As técnicas de processamento digital de sinal (DSP), como a conversão RGB $\rightarrow$ HSV e a aplicação do Filtro Passa-Alto (HPF) à vibração no próprio microcontrolador (módulos RGBSensor.cpp e ADXL335_Accelerometer.cpp), asseguraram que os dados brutos fossem transformados em *features* robustas antes de serem passadas para o modelo.

Eficiência e Coerência da Implementação TinyML

- **Otimização para *Edge Computing*:** A escolha por um modelo de Machine Learning de baixa complexidade (como o SVM ou MLP simplificado, Secção 6.2) e o uso da biblioteca TinyML (Eloquent) permitiram a serialização do modelo em *array* C (model_data.h) e seu armazenamento na memória Flash do ESP32. Esta abordagem garantiu que o sistema fosse eficiente em termos de memória e computação, adequando-se às severas restrições de um microcontrolador.
- **Garantia de Coerência:** O *pipeline* de inferência, explicitamente implementado na função runInference(), confirmou que o pré-processamento (aplicação do Z-Score exato via scaler_data.h) é aplicado no *Edge* antes da previsão. Esta coerência garante que a

precisão obtida no *dataset* de treino se mantenha no momento da operação em tempo real.

Arquitetura de Software

- A arquitetura modular do *firmware* facilitou o desenvolvimento, o *debugging* e a manutenção. O uso estratégico de um Web Server para disparar a inferência remotamente e visualizar resultados confirmou o potencial do sistema como um nó de Edge Computing de fácil acesso.

Trabalhos Futuros

Para aprimorar o sistema e expandir as suas capacidades, sugerem-se os seguintes caminhos para trabalhos futuros:

1. **Integração Completa do Sensor Indutivo:** Implementar o módulo de leitura ADC para o sensor de metal e realizar uma nova fase de treino (Fusão H , S , V , σ_{vib} , M) para validar o ganho de precisão na distinção de ligas metálicas, aumentando a robustez da classificação.
2. **Otimização de Energia:** Otimizar o código para gerir dinamicamente o consumo de energia. Isto incluiria a desativação da pilha Wi-Fi de forma mais robusta durante a aquisição de dados e a utilização de modos *Deep Sleep* entre as classificações.
3. **Expansão do *Dataset* e Classes:** Aumentar a diversidade do *dataset* de treino, incluindo mais classes de materiais (ex: Plástico, Cerâmica) e variações dentro de cada classe, para aumentar a generalização do modelo.